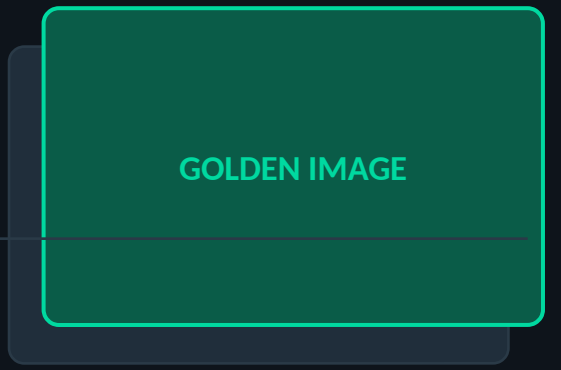
PHASE 2 · IMMUTABLE INFRASTRUCTURE

Packer: Golden Image Creation with VirtualBox

DevSecOps Lab Book · Chapter 07



HashiCorp Packer · VirtualBox · Cloud-Init · OVA Appliances

CHAPTER GOAL

From Manual Builds to Automated Golden Images

In Chapter 01, every Ubuntu VM was built by hand in VirtualBox.

That works — but it's slow, error-prone, and impossible to reproduce exactly.

This chapter automates image creation with HashiCorp Packer.

Packer will automatically:

- Download the Ubuntu ISO & create a temp VM
- Install Ubuntu, configure the OS, add packages
- Apply hardening, clean up, export an OVA

THE RESULT

Golden Image (OVA Appliance)

Import it once — clone it as many times as your lab needs. Every VM starts identical.

LEARNING OBJECTIVES

What You Will Be Able To Do

1. Understand immutable infrastructure
2. Understand Golden Images
3. Install HashiCorp Packer
4. Build VirtualBox images automatically
5. Perform unattended Ubuntu installation
6. Automate server provisioning
7. Export reusable OVA appliances
8. Version-control image definitions
9. Produce reproducible VM images

Twelve VMs, Fourteen Manual Steps

BUILDING ONE VM MANUALLY

- Create VM, allocate CPU & RAM
- Create disk, mount ISO, install Ubuntu
- Configure networking, SSH, firewall
- Install Docker, apply hardening
- Shut down and clone the VM

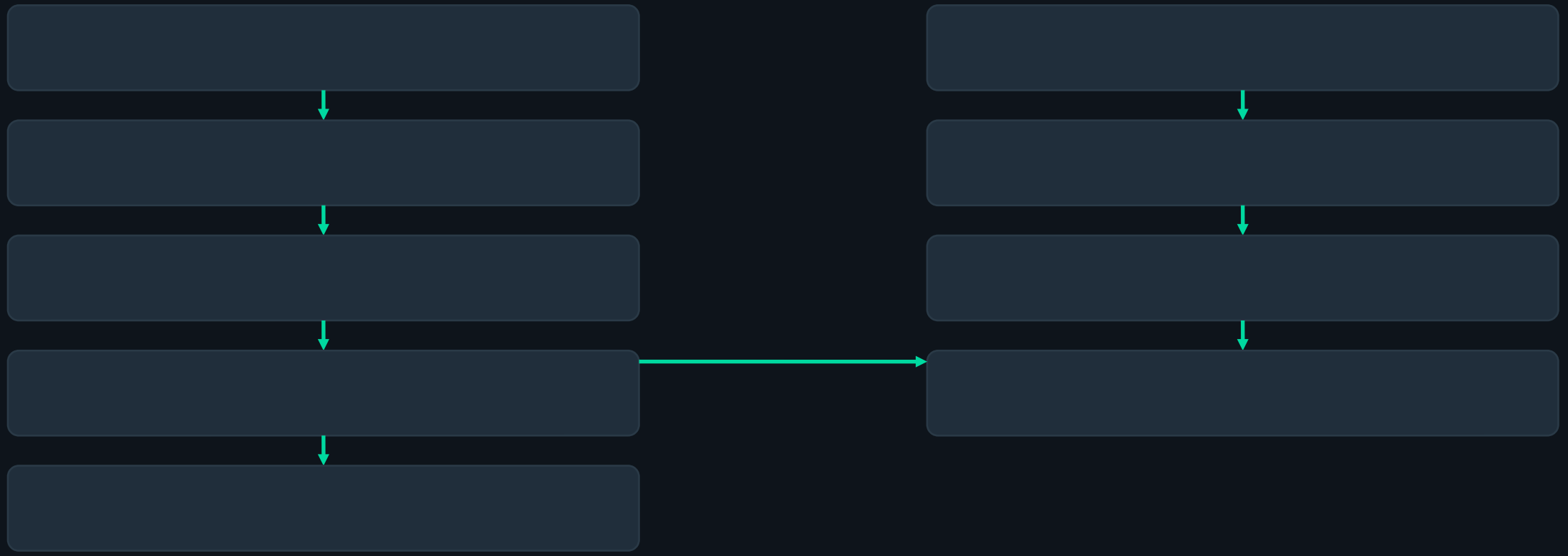
Now Ubuntu ships a security update — repeat all fourteen steps again, on every VM.

THE PROBLEM

- ✗ Human mistakes
- ✗ Different configuration on each VM
- ✗ Time consuming
- ✗ Difficult to audit
- ✗ Impossible to reproduce exactly
- ✗ Hard to update

If one admin forgets one command, every clone inherits that mistake.

Describe the Image as Code, Not Documentation



Every image is built exactly the same way — the process becomes deterministic.

One Install, Infinite Clones

A Golden Image is a fully prepared OS image that already contains:

- Ubuntu Server + updates & security patches
- SSH configuration
- Required packages & monitoring tools
- Docker prerequisites
- Time synchronization
- Security hardening

Install Ubuntu once →

clone the image as many times as needed.

This dramatically reduces deployment time compared to installing Ubuntu twelve separate times.

Replace, Don't Repair

THE OLD WAY

- Login to the server
- Install packages
- Edit configuration
- Hope nothing breaks

THE PACKER WAY

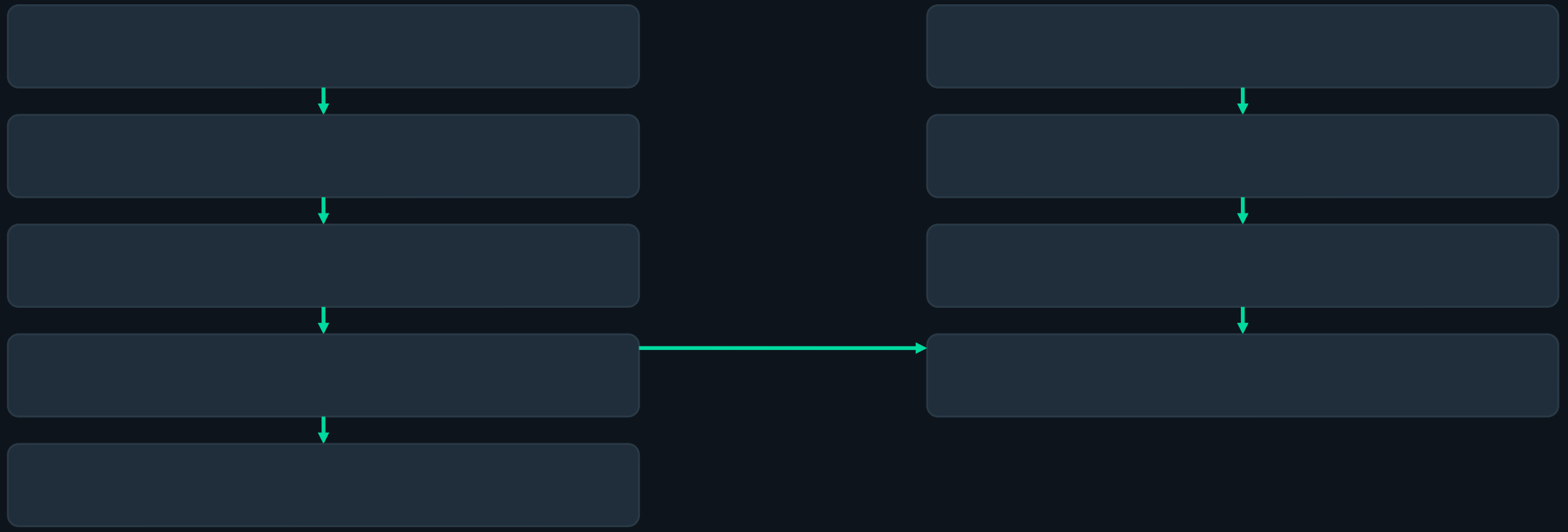
- Build a new image
- Test the image
- Replace the old VM

Modern DevOps discourages changing servers by hand. Packer is one of the main tools used to build immutable images.

The Vocabulary You'll Use Throughout

Builder	Creates the virtual machine
Source	Operating system installation source
Provisioner	Executes configuration scripts
Variable	User configurable value
Build	Complete image creation workflow
Post Processor	Exports or modifies the final artifact
Artifact	Final generated image

From ISO to Exported Appliance



The temporary VM is destroyed after export — only the OVA appliance remains.

VirtualBox, Packer, and the Plugin

1 VirtualBox

```
sudo apt update
sudo apt install \
  virtualbox \
  virtualbox-ext-pack

VBoxManage --version
```

VERIFY OUTPUT

```
7.1.12r169651
```

2 Packer

```
wget https://releases.\
hashicorp.com/packer/\
1.14.0/packer_1.14.0_\
linux_amd64.zip
unzip packer_*.zip
sudo mv packer /usr/\
local/bin/
```

VERIFY OUTPUT

```
Packer v1.14.0
```

3 VirtualBox Plugin

```
packer init .

# downloads the
# VirtualBox plugin
# automatically
```

Keep Provisioning Scripts Outside the Template

```
packer/
├── ubuntu-22.04.pkr.hcl
├── variables.pkr.hcl
├── http/
│   ├── user-data
│   └── meta-data
├── scripts/
│   ├── install-packages.sh
│   ├── hardening.sh
│   ├── cleanup.sh
│   └── validate.sh
└── output/
```

`ubuntu-22.04.pkr.hcl`

Main template — builder & build

`variables.pkr.hcl`

Configurable values (CPU, RAM, ISO…)

`http/`

Cloud-Init autoinstall files

`scripts/`

Provisioning, hardening & cleanup

`output/`

Where the exported OVA lands

variables.pkr.hcl

```
variable "vm_name" {
  default = "ubuntu-22.04-base"
}

variable "cpus"      { default = 2 }
variable "memory"    { default = 4096 }
variable "disk_size" { default = 20480 }

variable "ssh_username" { default = "ubuntu" }
variable "ssh_password" { default = "ubuntu" }

variable "iso_url" {
  default = "https://releases.ubuntu.com/
    22.04/ubuntu-22.04.5-live-server-amd64.iso"
}
```

WHY VARIABLES?

- One place to tune CPU, RAM & disk size
- Reusable across CI/CD pipelines
- Keeps secrets & sizing out of the template body
- ISO checksum must match the published SHA256

ubuntu-22.04.pkr.hcl

```
packer {
  required_plugins {
    virtualbox = {
      source  = "github.com/hashicorp/virtualbox"
      version = ">= 1.1.1"
    }
  }
}

source "virtualbox-iso" "ubuntu" {
  guest_os_type = "Ubuntu_64"
  vm_name   = var.vm_name
  cpus      = var.cpus
  memory    = var.memory
  disk_size = var.disk_size
  headless  = true
  ...
}
```

```
build {
  sources = [
    "source.virtualbox-iso.ubuntu"
  ]

  provisioner "shell" {
    script = "scripts/install-packages.sh"
  }
  provisioner "shell" {
    script = "scripts/hardening.sh"
  }
  provisioner "shell" {
    script = "scripts/cleanup.sh"
  }
}
```

Cloud-Init Autoinstall + Boot Command

http/meta-data

```
instance-id: ubuntu-template
local-hostname: ubuntu-template
```

http/user-data (excerpt)

```
#cloud-config
autoinstall:
  version: 1
  identity:
    hostname: ubuntu-template
    username: ubuntu
  ssh: { install-server: true }
```

BOOT COMMAND POINTS AT CLOUD-INIT

```
boot_command = [
  "<esc><wait>",
  "e<wait>",
  " autoinstall ds=nocloud-net;",
  "  s=http://{{.HTTPIP}}:{{.HTTPPort}}/",
  "<f10>"
]
```

Packer serves user-data & meta-data over a temporary HTTP server.

Ubuntu downloads them and installs completely unattended — no keyboard interaction required.

⚠️ *Never store plaintext passwords — generate a hash with openssl passwd -6*

Install Packages, Then Harden

scripts/install-packages.sh

curl · wget · git · vim · jq · net-tools · dnsutils · tcpdump · nmap · python3 · chrony · fail2ban · auditd · ufw …

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
  curl wget git vim jq \
  net-tools dnsutils tcpdump \
  python3 python3-pip \
  chrony fail2ban auditd ufw
echo "Packages installed successfully."
```

scripts/hardening.sh

```
sudo timedatectl set-timezone UTC
sudo systemctl enable chrony auditd fail2ban
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw --force enable
```

SSH hardening (sshd_config.d/hardening.conf):

- PermitRootLogin no · MaxAuthTries 3
- PubkeyAuthentication yes · X11Forwarding no

Erase Identity Before Exporting

```
sudo apt autoremove -y
sudo apt clean
sudo rm -rf /tmp/* /var/tmp/*

sudo truncate -s 0 /etc/machine-id
sudo rm -f /var/lib/dbus/machine-id
sudo ln -s /etc/machine-id /var/lib/dbus/machine-id

sudo rm -f /etc/ssh/ssh_host_*
history -c
```

WHY THIS MATTERS

Removing the machine-id and SSH host keys ensures every clone generates its own unique identity on first boot.

Without this step, every cloned VM would share the same machine-id and SSH fingerprints — a serious fleet-management problem.

packer build . (≈ 10–20 Minutes)

1 · VALIDATE

```
packer init .
packer fmt .
packer validate .
```

✓ The configuration is valid.

2 · BUILD

```
packer build .
```

- Download ISO → boot installer
- Cloud-init installation
- SSH in, provision, harden
- Cleanup → shutdown → export

3 · OUTPUT

Build finished successfully.

```
output/
ubuntu-22.04-base.ova
```

This OVA file is your Golden Image.

Why Distribute an OVA?

VERIFY & IMPORT

```
ls -lh output          # ubuntu-22.04-base.ova
(2.4G)
VBoxManage import output/ubuntu-22.04-base.ova
VBoxManage list vms    # "ubuntu-22.04-base"
```

GUI alternative: VirtualBox → File → Import Appliance → browse to the .ova → Next → Finish.

WHY AN OVA APPLIANCE?

- Portable & platform independent
- Version controlled
- Easy backup & distribution
- Importable into any VirtualBox install

Instead of rebuilding Ubuntu on every workstation, simply distribute the OVA file.

One Image → Twelve Purpose-Built VMs

dns-01 DNS Server	**idm-01** Identity Management	**gitlab-01** GitLab
harbor-01 Harbor Registry	**vault-01** HashiCorp Vault	**runner-01** GitLab Runner
monitor-01 Prometheus	**monitor-02** Grafana	**logs-01** Loki
kube-master-01 Kubernetes	**kube-worker-01** Kubernetes	**kube-worker-02** Kubernetes

Full Clone vs. Linked Clone

FULL CLONE ✓ recommended

- Completely independent
- Safe to delete the original
- Uses more disk, slower to create

LINKED CLONE

- Extremely fast, minimal storage
- Depends on the parent VM
- Parent cannot be removed

```
VBoxManage clonevm \
  "ubuntu-22.04-base" \
  --name gitlab-01 \
  --register
```

RESOURCE SIZING (VBoxManage modifyvm)

```
modifyvm gitlab-01 --memory 8192 --cpus 4
modifyvm kube-master-01 --memory 8192 --cpus 4
modifyvm kube-worker-01 --memory 6144 --cpus 4
```

NETWORKING (Internal Network)

```
modifyvm dns-01 --nic1 intnet \
  --intnet1 DevSecOps-Lab
# temp internet: --nic2 nat, then remove
```

Confirm Every Clone Is Healthy

ON FIRST BOOT AFTER CLONING

- A new machine-id is generated
- New SSH host keys are created
- Networking initializes
- Cloud-init runs first-boot tasks

```
hostnamectl
ls /etc/ssh/   # fresh host keys
```

VALIDATE THE IMAGE

```
cat /etc/os-release
hostname
sudo ufw status
sudo systemctl status ssh
sudo systemctl status fail2ban
sudo systemctl status auditd
timedatectl
```

Never Patch VMs by Hand



When Ubuntu releases new security patches, do not update every VM manually. Rebuild the Golden Image and redeploy — this keeps every deployment identical.

Four Habits That Keep Golden Images Trustworthy

1 Keep Templates Small

Install only packages required by every server — avoid application-specific software.

2 Version Everything

Store the Packer project in Git: git init · git add . · git commit.

3 Tag Releases

v1.0 initial build · v1.1 security updates · v1.2 Docker · v2.0 OS migration.

4 Never Modify the Template VM

Treat the Golden Image as read-only — modify the template, rebuild, export again.

Common Failures and Their Fixes

Packer can't find VirtualBox

Verify VBoxManage --version; install if missing.

VT-x / AMD-V disabled

Enable hardware virtualization in BIOS/UEFI.

Build stops waiting for SSH

Check username, password, cloud-init logs, SSH service.

ISO checksum mismatch

Ubuntu republished the ISO — update iso_checksum.

OVA export failed

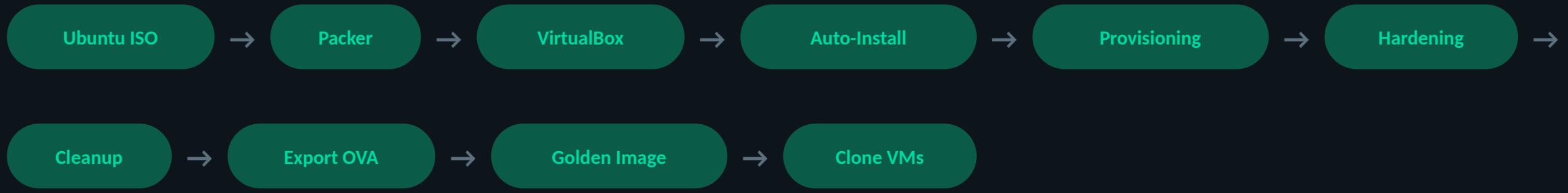
Check disk space (df -h) and output directory permissions.

Appliance import failed

Re-verify the OVA; rebuild the image if necessary.

7.36 — CHAPTER SUMMARY

One Golden Image. Every VM in the Lab.



THIS GIVES YOU

- Reproducible infrastructure
- Faster deployments
- Consistent configurations
- Version-controlled image definitions

The Golden Image built in this chapter becomes the foundation for every virtual machine in the DevSecOps laboratory.